



FundChain

Decentralized Crowdfunding Application

Built as a Beginner Blockchain Project

1. What is a DApp? Starting from Zero

Before we dive into FundChain, let's understand the basics from

1.1 The Traditional Web — How Apps Work Today

When you use an app like Kickstarter or GoFundMe, here is what happens behind the scenes:



Analogy: The Bank Model

Imagine depositing money in a bank. You trust the bank to keep your money safe, record your balance correctly, and transfer it when needed. The bank is the middleman — you cannot move your money without going through them.

In traditional apps:

- Your data lives on the company's private servers
- The company controls all the rules and can change them
- You must trust the company to be honest
- If the company shuts down, your data and funds are at risk
- The company takes a % fee from every transaction

1.2 Blockchain — The New Model

A blockchain is a shared digital ledger that thousands of computers around the world maintain together. No single person or company owns it.

📖 Analogy: The Public Notebook

Imagine a notebook that 10,000 people all have identical copies of. When you write something new, everyone's copy updates simultaneously. Nobody can secretly change what's written because everyone else's copy would not match. This is the core idea of blockchain.

✗ Traditional App

Central server
Company controls data
Fees taken by platform
Can be shut down
Trust the company

☑ DApp on Blockchain

Distributed network
Code controls data
No platform fees
Always available
Trust the code

1.3 Smart Contracts — The Brain of a DApp

A smart contract is a program that lives on the blockchain. It automatically enforces rules without needing a human or company to manage it.

⚡ Analogy: The Vending Machine

A vending machine is like a smart contract. You insert money, press a button, and the machine automatically gives you the product — no cashier needed. The rules are built into the machine. FundChain's smart contract works the same way: when someone donates ETH, the contract automatically records it and holds the funds according to pre-set rules.

2. FundChain — Project Overview

FundChain is a Crowdfunding Decentralized Application (DApp) built on the Ethereum blockchain. It allows users to create fundraising campaigns and receive contributions directly — without any middleman, bank, or platform taking fees.

💡 In Simple Terms

FundChain is like crowdfunding app, but the entire platform runs on code — not on a company's server. Every donation, every campaign, every rule is managed by a Smart Contract on the blockchain. It's transparent, unstoppable, and trustless.

2.1 What FundChain Can Do

#	Feature	Description
1	Create Campaign	Anyone can launch a fundraiser with a title, description, and ETH goal
2	Donate ETH	Any user can contribute ETH to any active campaign
3	Withdraw Funds	Campaign owner can withdraw once the funding goal is reached
4	Progress Tracking	Visual progress bar shows % of goal reached in real time
5	Live Statistics	Dashboard shows total campaigns, active count, and total ETH raised
6	Wallet Connection	MetaMask integration identifies each user by their wallet address
7	Toast Notifications	On-screen alerts for every transaction success or failure

2.2 Important: No Real Money Used



Safe to Use

This project runs entirely on a LOCAL blockchain simulated by Ganache on your own computer. All ETH used is fake test ETH — it has no real-world value and cannot be transferred to the real Ethereum network. This makes it completely safe to experiment with.

3. Tools & Technologies

FundChain uses 7 carefully chosen tools. Each plays a specific role. Here is every tool explained simply:

Tool	Role	What it does in FundChain
Solidity	Smart Contract Language	Writes the rules of the DApp — donation logic, ownership checks, fund storage. Runs on Ethereum.
MetaMask	Crypto Wallet (Browser)	Acts as your identity and wallet. Signs every transaction so the blockchain knows who sent it.
Ganache	Local Fake Blockchain	Creates a personal Ethereum blockchain on your PC with 10 fake accounts, each having 100 test ETH.
Truffle	Compile & Deploy Tool	Compiles Solidity code into bytecode, then deploys it to the Ganache blockchain.
React	Frontend Framework	Builds the user interface — the buttons, cards, forms, and progress bars you see in the browser.
ethers.js	Blockchain Bridge	JavaScript library that lets React talk to the smart contract — sends transactions, reads data.
ABI (JSON)	Contract Interface	A description file that tells ethers.js what functions the smart contract has and how to call them.

3.1 How the Tools Relate to Each Other

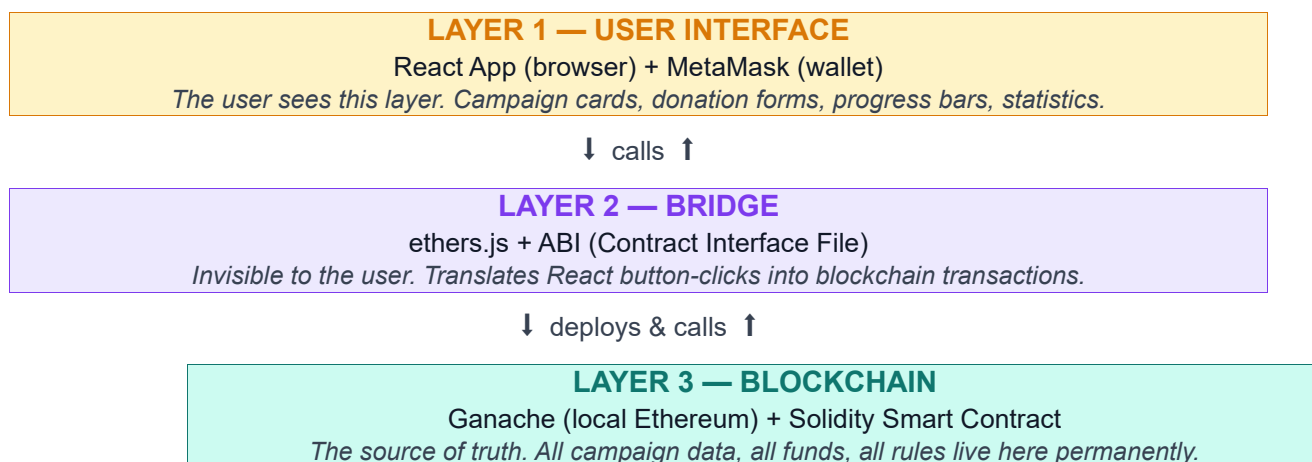
Think of the tools in three layers:

BLOCKCHAIN LAYER	Ganache (local chain) + Solidity (smart contract) + Truffle (deploy tool)
BRIDGE LAYER	ethers.js + ABI file — connects the frontend to the blockchain
USER LAYER	React (what you see) + MetaMask (your identity/wallet)

4. System Architecture

Architecture means how all the parts of the system are arranged and how they communicate. FundChain has three distinct layers that work together.

4.1 The Three-Layer Architecture



4.2 Project Folder Structure

Here is how the files are organized on computer:

File / Folder	What it does
<code>contracts/Crowdfunding.sol</code>	The smart contract — all the blockchain logic lives here
<code>migrations/1_deploy.js</code>	Instructions telling Truffle how to deploy the contract to Ganache
<code>truffle-config.js</code>	Configuration: connects Truffle to Ganache (IP address + port)
<code>frontend/src/App.js</code>	Main React file — the entire user interface
<code>frontend/src/contract.json</code> (ABI)	Auto-generated interface file that lets ethers.js call the contract
<code>ganache workspace</code>	Running locally on your PC — the fake blockchain

5. The Smart Contract — Crowdfunding.sol

The smart contract is the heart of FundChain. It is written in Solidity and runs on the Ethereum blockchain (Ganache locally). Once deployed, nobody — not even the developer — can change its rules.

5.1 The Campaign Data Structure

A 'struct' in Solidity is like a form template. Every campaign created on FundChain stores these fields:

Field	Data Type	What it stores
<code>owner</code>	<code>address</code>	Wallet address of the person who created the campaign
<code>title</code>	<code>string</code>	Name of the campaign (e.g. 'Education Fund')
<code>description</code>	<code>string</code>	Details about what the campaign is for
<code>goal</code>	<code>uint256</code>	Target amount in Wei (smallest ETH unit)
<code>amountRaised</code>	<code>uint256</code>	Total ETH donated so far — updates with each donation
<code>withdrawn</code>	<code>bool</code>	True/False — has the owner already withdrawn funds?

5.2 The Four Functions

The contract has 4 functions. Each does exactly one job:

🔗 `createCampaign(title, description, goal)`

Called when a user launches a new campaign. Takes the title, description and funding goal as inputs. Creates a new Campaign entry and stores it permanently on the blockchain. Any wallet address can call this.

💖 `donate(campaignId)`

Called when a user clicks 'Fund'. The user sends real ETH with this call (marked 'payable'). The contract checks the campaign exists, then adds the sent amount to `amountRaised`. The funds are held inside the contract itself — not in anyone's wallet.

**withdraw(campaignId)**

Called by the campaign owner to claim raised funds. The contract runs THREE security checks: (1) Is the caller the owner? (2) Has the goal been reached? (3) Has this already been withdrawn? Only if all three pass does the contract transfer ETH to the owner's wallet.

**getCampaigns()**

A read-only function (costs no gas/ETH). Returns all campaigns stored on the blockchain. React calls this every time the page loads or refreshes, to display the campaign cards.

6. Full System Workflow

This section walks through exactly what happens — step by step — from the moment a user opens the browser to the moment a campaign owner withdraws funds. Nothing is skipped.

6.1 Startup Workflow

①	Open App <i>User</i>	User visits localhost:3000. React loads in the browser. The page checks if MetaMask is installed.
②	Connect Wallet <i>User + MetaMask</i>	User clicks 'Connect Wallet'. MetaMask popup appears asking permission. User approves. MetaMask shares the wallet address with React.
③	Load Campaigns <i>React + ethers.js</i>	ethers.js calls getCampaigns() on the smart contract. Contract returns all stored campaigns. React renders them as cards on screen.

6.2 Creating a Campaign

①	Fill Form <i>User</i>	User types campaign title, description, and funding goal (in ETH) into the form on the left side of the screen.
②	Click Launch <i>React</i>	React reads the form values and calls the <code>createCampaign()</code> function via <code>ethers.js</code> , converting ETH to Wei (1 ETH = 1,000,000,000,000,000,000 Wei).
③	MetaMask Confirms <i>MetaMask</i>	MetaMask intercepts the transaction and shows a popup: 'FundChain wants to create a transaction'. User reviews the gas fee and clicks Confirm.
④	Blockchain Records <i>Ganache + Contract</i>	The transaction is sent to Ganache. The smart contract executes <code>createCampaign()</code> , pushes a new Campaign struct into the <code>campaigns</code> array. Stored permanently.
⑤	UI Updates <i>React</i>	React shows a success toast notification. <code>getCampaigns()</code> is called again. The new campaign card appears on screen.

6.3 Donating to a Campaign

①	Enter Amount <i>User</i>	User types an ETH amount into the input field on a campaign card and clicks the green 'Fund' button.
②	Transaction Built <i>ethers.js</i>	<code>ethers.js</code> creates a transaction object: <code>{ to: contractAddress, value: enteredETH, data: donate(campaignId) }</code> . This is sent to MetaMask.
③	User Signs <i>MetaMask</i>	MetaMask shows the transaction details. User clicks Confirm. MetaMask signs the transaction with the user's private key — this proves identity.

④	ETH Transferred <i>Ganache + Contract</i>	Ganache receives the signed transaction. Smart contract runs <code>donate()</code> : adds the ETH amount to <code>campaign.amountRaised</code> . ETH is now held inside the contract.
⑤	Progress Updates <i>React</i>	React refreshes. The campaign card now shows the updated progress bar and raised amount. If goal is reached, the Withdraw button becomes active.

6.4 Withdrawing Funds

①	Owner Clicks Withdraw <i>Campaign Owner</i>	Only the wallet that created the campaign sees the active Withdraw button. Owner clicks it.
②	Contract Checks <i>Smart Contract</i>	Three automatic security checks run: Is caller the owner? Is goal reached? Already withdrawn? If any check fails, transaction is rejected with an error message.
③	ETH Released <i>Contract → Owner Wallet</i>	All checks pass. Contract executes: <code>payable(owner).transfer(amountRaised)</code> . ETH moves from the contract to the owner's MetaMask wallet.
④	Campaign Closed <i>Ganache</i>	The 'withdrawn' field is set to true. The campaign cannot be withdrawn from again — even by the owner.

7. Security Features

Even though this is a beginner project using fake ETH, the security logic is identical to production blockchain applications. The smart contract has three built-in security checks using Solidity's `require()` statement.



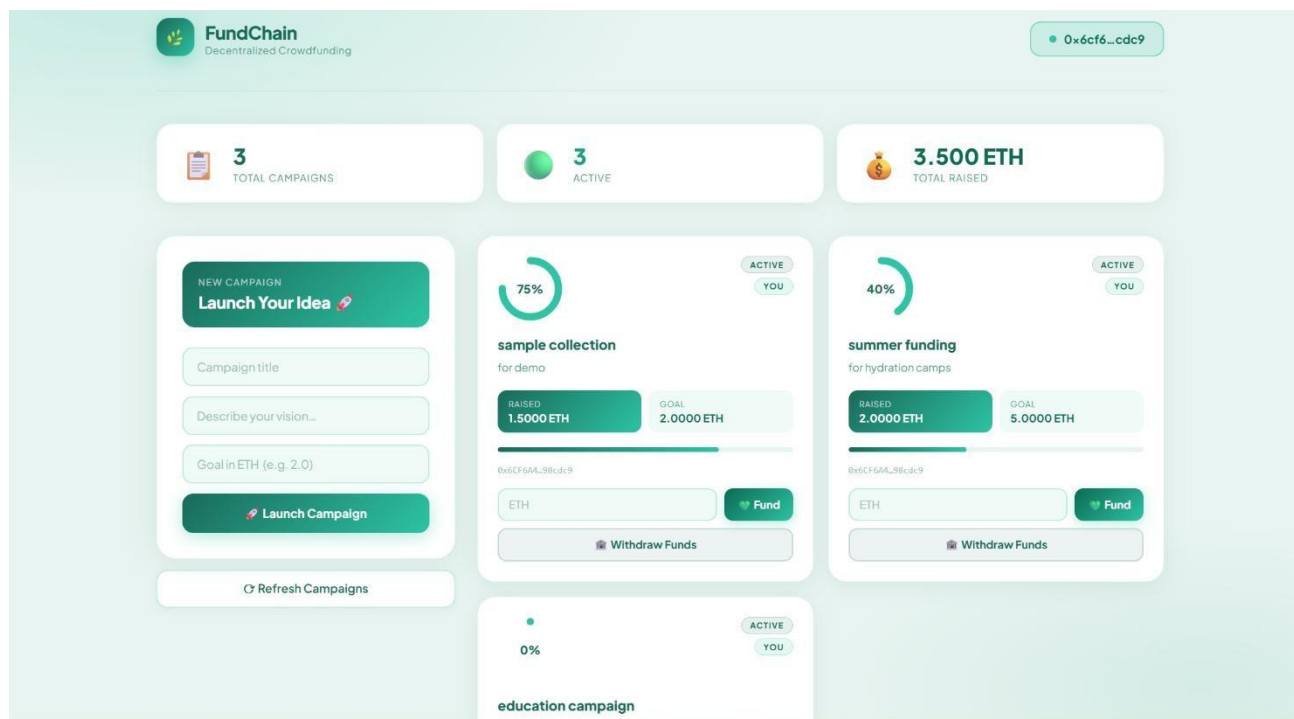
How require() Works

require() is Solidity's guard clause. It takes a condition and an error message. If the condition is FALSE, the entire transaction is immediately cancelled and the error message is returned. No ETH is moved. No state is changed.

Security Check	Location	What it prevents
Campaign must exist	<code>donate()</code>	Donating to a campaign ID that does not exist (would cause a crash)
Only owner can withdraw	<code>withdraw()</code>	Anyone other than the creator stealing the raised funds
Goal must be reached	<code>withdraw()</code>	Owner withdrawing before the funding target is met
Cannot withdraw twice	<code>withdraw()</code>	Owner draining the contract multiple times (double-spend attack)

8. Frontend

The frontend is built with React and runs in browser at localhost. It is the visual layer — the beautiful green-themed interface you see when you open the app. FundChain uses a clean teal/green color palette that gives it a professional, trustworthy feel.



8.1 UI Components Breakdown

Component	What it does
Top Navigation Bar	Shows the FundChain logo and the connected wallet address (e.g. 0x6cf6...cdc9) in the top right corner
Statistics Panel	3 cards at the top: Total Campaigns, Active Campaigns, Total ETH Raised. Updates in real time.
Create Campaign Form	Left panel with 3 inputs (title, description, goal) and the green 'Launch Campaign' button
Campaign Cards	Each active campaign shows: title, description, circular progress chart, RAISED/GOAL amounts, donate input, Fund button, Withdraw button
Progress Circle	Circular chart showing % of goal reached — visual at a glance (e.g. 75%, 40%, 0%)
ACTIVE Badge	Green badge on each campaign showing it is still open for donations
YOU Badge	Teal badge shown only on campaigns created by the currently connected wallet

Toast Notifications	Pop-up alerts at the corner of the screen: 'Campaign created!', 'Donation successful!', 'Funds withdrawn!'
Refresh Button	Manually re-fetches all campaigns from the blockchain and updates the display

9. Key Concepts Summary

A quick-reference glossary of every important term used in this project:

Term	Plain English Explanation
Blockchain	A chain of data blocks shared across thousands of computers. Once written, data cannot be changed.
DApp	Decentralized Application — an app whose backend runs on a blockchain instead of a company's server.
Smart Contract	A self-executing program stored on the blockchain. Runs automatically when conditions are met — no human needed.
Solidity	The programming language used to write smart contracts on Ethereum. Similar to JavaScript.
Ethereum	The most popular blockchain platform for building DApps and smart contracts.
Ganache	A fake local Ethereum blockchain that runs on your PC for development and testing.
MetaMask	A browser extension wallet. Manages your blockchain identity and signs transactions.
Truffle	A toolkit that compiles Solidity code and deploys smart contracts to a blockchain.
ethers.js	A JavaScript library that lets web apps (React) communicate with the Ethereum blockchain.
ABI	Application Binary Interface — a JSON file describing the smart contract's functions so JavaScript can call them.
ETH / Wei	ETH is Ethereum's currency. Wei is the smallest unit: 1 ETH = 1,000,000,000,000,000,000 Wei.
Gas	A small fee paid for every blockchain transaction to compensate the network. In Ganache, it's fake.

Transaction	Any action that changes blockchain state (creating a campaign, donating, withdrawing). Must be signed.
Wallet Address	A unique 42-character identifier (0x...) representing a user on the blockchain — like a bank account number.
Private Key	A secret cryptographic key that proves you own a wallet. NEVER share this with anyone.
require()	Solidity's security guard. Stops a transaction and reverts all changes if a condition is not met.
msg.sender	Built-in Solidity variable containing the wallet address of whoever called the function.
msg.value	Built-in Solidity variable containing the amount of ETH sent with a transaction.
payable	Keyword in Solidity that allows a function to receive ETH. Without it, ETH sent will be rejected.
struct	A custom data type in Solidity — like a form template with multiple named fields.

10. Conclusion

FundChain is a complete, working demonstration of how blockchain technology can replace traditional centralized platforms for fundraising. Built entirely with free, open-source tools, it proves that decentralized applications are not just theoretical — they can be built, run, and demonstrated by a beginner.

What Was Achieved

	A working smart contract deployed to a local blockchain with real Solidity security logic
	A professional React frontend with real-time blockchain data, MetaMask integration, and a beautiful UI
	Full end-to-end flow: create campaign → donate ETH → track progress → withdraw funds
	Zero real money used — entirely safe, local, and free
	Production-grade patterns: ownership checks, double-spend prevention, event-driven UI updates

What This Project Teaches

- How blockchains store data permanently and transparently
- How smart contracts enforce rules without any human oversight
- How a frontend communicates with a blockchain using ethers.js
- How MetaMask acts as both an identity provider and a transaction signer
- The full development workflow: write → compile → deploy → connect → test

Next Steps to Grow This Project

To take FundChain further, you could: (1) Deploy to a real public testnet like Sepolia for free using Infura. (2) Add campaign deadlines using block timestamps. (3) Add a refund mechanism if the goal is not reached by the deadline. (4) Add IPFS for storing campaign images decentrally. (5) Add events to the smart contract for better frontend updates.



Built from scratch. Runs on blockchain. Zero middlemen.